

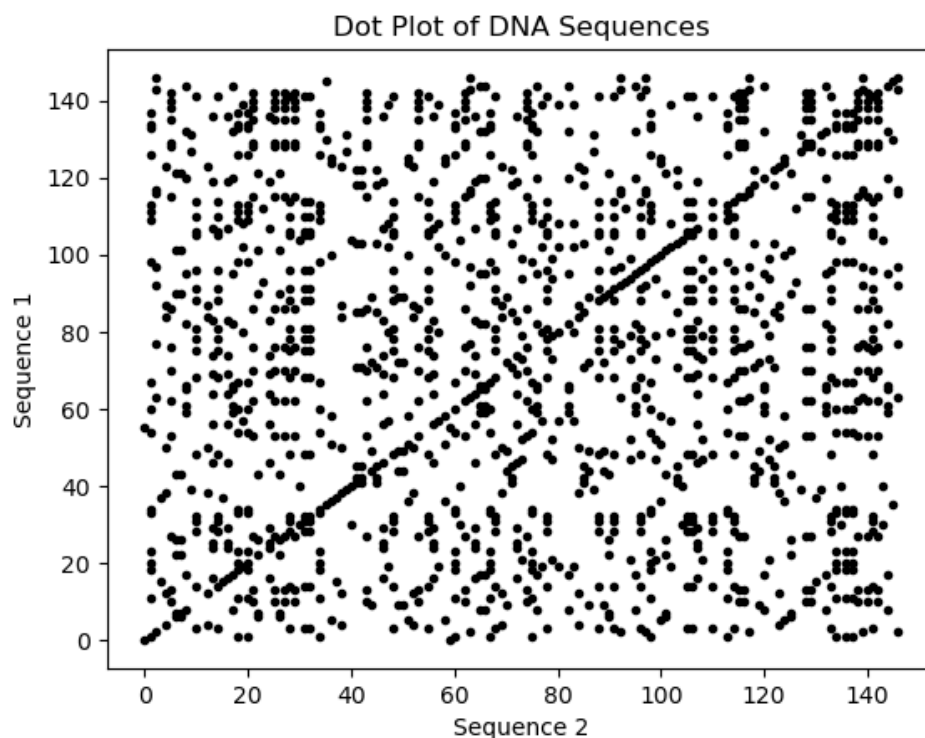
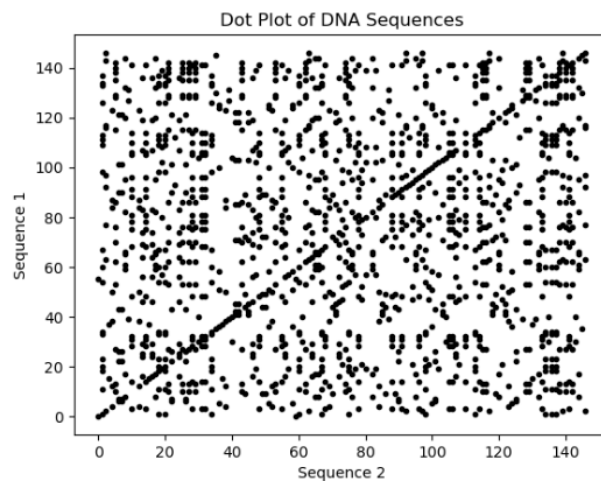
Question 1

The beta chain sequences of hemoglobin for both Human and chicken species were acquired from UniProtKB

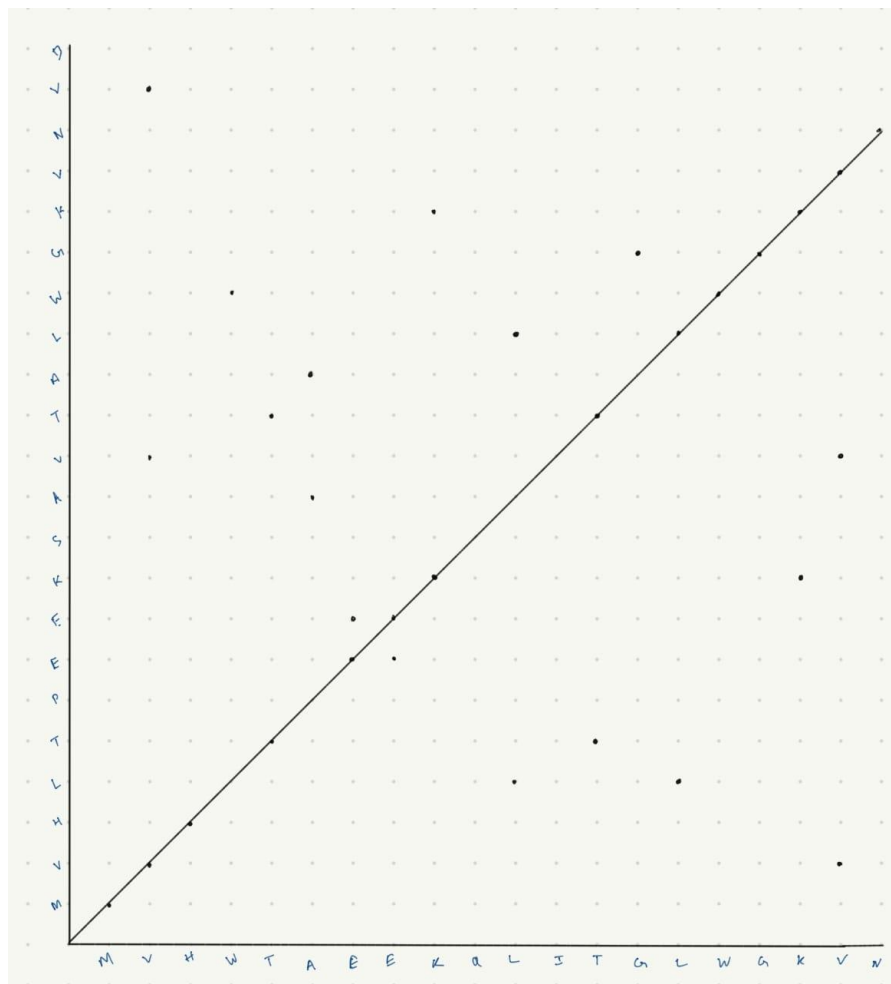
```
In [2]: import matplotlib.pyplot as plt
```

```
In [9]: sequence1='MVHLTPEEKSAVTALWGKVNVDEVGGEALGRLLVVYPWTQRFFESFGDLSTPDAMVGNPKVKAHGKKVLGAFSDGLAHLNLTGTFTATLSELHCDKLVHPDENFRLLGNVLVCLAHF  
sequence2='MVHWTAEKQLITGLWGKVNVAECCGAEALARLLIVYPWTQRFFASFGNLSSTAILGNPMVRAHGKKVLTSFGDAVKNLNDIKNTFSQSELHCDKLVHPDENFRLLGDILIVLAAP'
```

```
In [10]: import matplotlib.pyplot as plt  
  
dot_plot_data = [[1 if sequence1[i] == sequence2[j] else 0 for j in range(len(sequence2))] for i in range(len(sequence1))]  
  
# Plotting the dot plot  
for i in range(len(sequence1)):  
    for j in range(len(sequence2)):  
        if dot_plot_data[i][j] == 1:  
            plt.scatter(j, i, c='black', marker='.')  
  
plt.xlabel('Sequence 2')  
plt.ylabel('Sequence 1')  
plt.title('Dot Plot of DNA Sequences')  
plt.show()
```



The program is verified manually



Common segments:

```
In [33]: diag_elements= np.diagonal(dot_plot_data)
segments=''.join(sequence1[i] if diag_elements[i]==1 else " " for i in range(len(diag_elements)-1)).split()

if diag_elements[-1]==1:
    segments[-1]+=sequence1[-1]

print("Common segments",segments)
```

Common segments ['MVH', 'T', 'EEK', 'T', 'LWGKVV', 'E', 'G', 'EAL', 'RLL', 'VYPWTQRFF', 'SFG', 'LS', 'P', 'A', 'GNP', 'V', 'AH', 'GKKVL', 'F', 'D', 'LDN', 'K', 'TF', 'LSELHCDKLHVDPENFRLLG', 'L', 'VLA', 'HF', 'K', 'FTP', 'QAA', 'QK', 'V', 'VA', 'ALA', 'KYH']

Question 2

```
In [55]: def score(seq1, seq2):
    match = 0
    mismatch = 0
    length_penalty = 0
    origination_penalty = 0
    n = len(seq1)

    for i in range(n):
        if seq1[i] == seq2[i]:
            match += 1
        elif seq1[i] == '-' or seq2[i] == '-':
            length_penalty += 1
            origination_penalty += 1
            if i > 0 and (seq1[i - 1] == '-' or seq2[i - 1] == '-'):
                origination_penalty -= 1
        elif seq1[i] != seq2[i]:
            mismatch += 1

    score = (1 * match) + (0 * mismatch) + (-1 * length_penalty) + (-2 * origination_penalty)
    return score
```

```
In [56]: score('AATCTATA', 'AAG--ATA')
```

```
Out[56]: 1
```

Question 3

Verify Q2 manually

Q3 A A T C T A T A
 A A G - - A T A

$$\text{Score} = 5(1) + 1 \times 0 + 1(-2) + 2(-1)$$

$$\text{Score} = 5 - 4$$

$$\text{Score} = 1$$

Question 4

```
In [7]: def needleman_wunsch(seq1, seq2, match_score=2, mismatch_penalty=-1, gap_penalty=-2):
# Initialization
rows, cols = len(seq1) + 1, len(seq2) + 1
edit_matrix = [[0] * cols for _ in range(rows)]

for i in range(1, rows):
    edit_matrix[i][0] = i * gap_penalty
for j in range(1, cols):
    edit_matrix[0][j] = j * gap_penalty
for i in range(1, rows):
    for j in range(1, cols):
        if seq1[i - 1] == seq2[j - 1]:
            match = edit_matrix[i - 1][j - 1] + match_score
        else:
            match = edit_matrix[i - 1][j - 1] + mismatch_penalty
        horz = edit_matrix[i - 1][j] + gap_penalty
        vert = edit_matrix[i][j - 1] + gap_penalty
        edit_matrix[i][j] = max(match, horz, vert)

aligned_seq1, aligned_seq2 = '', ''
i, j = rows - 1, cols - 1
while i > 0 or j > 0:
    if i > 0 and j > 0 and edit_matrix[i][j] == edit_matrix[i - 1][j - 1] + (match_score if seq1[i - 1] == seq2[j - 1] else mismatch_penalty):
        aligned_seq1 = seq1[i - 1] + aligned_seq1
        aligned_seq2 = seq2[j - 1] + aligned_seq2
        i -= 1
        j -= 1
    elif i > 0 and edit_matrix[i][j] == edit_matrix[i - 1][j] + gap_penalty:
        aligned_seq1 = seq1[i - 1] + aligned_seq1
        aligned_seq2 = '-' + aligned_seq2
        i -= 1
    else:
        aligned_seq1 = '-' + aligned_seq1
        aligned_seq2 = seq2[j - 1] + aligned_seq2
        j -= 1

print(edit_matrix)
return aligned_seq1, aligned_seq2
```

```
In [8]: needleman_wunsch('ACAGTCGAACG', 'ACCGTCGC')
```

```
[[0, -2, -4, -6, -8, -10, -12, -14, -16], [-2, 2, 0, -2, -4, -6, -8, -10, -12], [-4, 0, 4, 2, 0, -2, -4, -6, -8], [-6, -2, 2, 3, 1, -1, -3, -5, -7], [-8, -4, 0, 1, 5, 3, 1, -1, -3], [-10, -6, -2, -1, 3, 7, 5, 3, 1], [-12, -8, -4, 0, 1, 5, 9, 7, 5], [-14, -10, -6, -2, 3, 7, 8, 9], [-16, -12, -8, -4, 0, 1, 5, 6, 7], [-18, -14, -10, -6, -2, -1, 3, 4, 5], [-20, -16, -12, -8, -4, -3, 1, 5, 3], [-22, -18, -14, -10, -6, -5, -1, 3, 7]]
```

```
Out[8]: ('ACAGTCGAACG', 'ACCGTC---CG')
```

The alignment score is 7

The aligned sequences are:

ACAGTCGAACG

ACCGTC---CG

Question 5

Verify Q4 manually

X	-	A	C	C	G	T	C	C	G
-	0	-2	-4	-6	-8	-10	-12	-14	-16
A	-2	2	0	-2	-4	-6	-8	-10	-12
C	-4	0	4	2	0	-2	-4	-6	-8
A	-6	-2	2	3	1	-1	-3	-5	-7
G	-8	-4	0	1	5	3	1	-1	-3
T	-10	-6	-2	-1	3	7	5	3	1
C	-12	-8	-4	0	1	5	8	7	5
G	-14	-10	-6	-2	2	3	7	8	9
A	-16	-12	-8	-4	0	1	5	6	8
A	-18	-14	-10	-6	-2	-1	3	4	5
C	-20	-16	-12	-8	-4	-3	1	5	3
G	-22	-18	-14	-10	-6	-5	-1	3	7

From the above matrix :

A C A G T C G A A C G
A C C G T C - - - C G

$$\text{Score} = 7 \times 2 + 1(-1) + 3(-2)$$

$$\text{Score} = 7$$

Thus verified

Question 6

```
In [57]: def smith_waterman(seq1, seq2, match_score=2, mismatch_penalty=-1, gap_penalty=-2):
# Initialization
rows, cols = len(seq1) + 1, len(seq2) + 1
edit_matrix = [[0] * cols for _ in range(rows)]

max_score = 0
max_i, max_j = 0, 0

for i in range(1, rows):
    for j in range(1, cols):
        if seq1[i - 1] == seq2[j - 1]:
            match = edit_matrix[i - 1][j - 1] + match_score
        else:
            match = edit_matrix[i - 1][j - 1] + mismatch_penalty
        horz = edit_matrix[i - 1][j] + gap_penalty
        vert = edit_matrix[i][j - 1] + gap_penalty
        edit_matrix[i][j] = max(match, horz, vert, 0)

# Update the maximum score and its position
if edit_matrix[i][j] > max_score:
    max_score = edit_matrix[i][j]
    max_i, max_j = i, j

aligned_seq1, aligned_seq2 = '', ''
i, j = max_i, max_j
while i > 0 and j > 0 and edit_matrix[i][j] != 0:
    if edit_matrix[i][j] == edit_matrix[i - 1][j - 1] + (match_score if seq1[i - 1] == seq2[j - 1] else mismatch_penalty):
        aligned_seq1 = seq1[i - 1] + aligned_seq1
        aligned_seq2 = seq2[j - 1] + aligned_seq2
        i -= 1
        j -= 1
    elif edit_matrix[i][j] == edit_matrix[i - 1][j] + gap_penalty:
        aligned_seq1 = seq1[i - 1] + aligned_seq1
        aligned_seq2 = '-' + aligned_seq2
        i -= 1
    else:
        aligned_seq1 = '-' + aligned_seq1
        aligned_seq2 = seq2[j - 1] + aligned_seq2
        j -= 1

print(edit_matrix)
return max_score, aligned_seq1, aligned_seq2
```

```
In [58]: smith_waterman('ACGTATCGCGTATA', 'GATGCGTATCG')
```

```
[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 2, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0], [0, 0, 0, 1, 0, 2, 0, 0, 0, 1, 2, 0], [0, 2, 0, 0, 3, 1, 4, 2, 0, 0, 0, 4], [0, 0, 1, 2, 1, 2, 2, 6, 4, 2, 0, 2], [0, 0, 2, 0, 1, 0, 1, 4, 8, 6, 4, 2], [0, 0, 0, 4, 2, 0, 0, 3, 6, 10, 8, 6], [0, 0, 0, 2, 3, 4, 2, 1, 4, 8, 12, 10], [0, 2, 0, 0, 4, 2, 6, 4, 2, 6, 10, 14], [0, 0, 1, 0, 2, 6, 4, 5, 3, 4, 8, 12], [0, 2, 0, 0, 2, 4, 8, 6, 4, 2, 6, 10], [0, 0, 1, 2, 0, 2, 6, 10, 8, 6, 4, 8], [0, 0, 2, 0, 1, 0, 4, 8, 12, 10, 8, 6], [0, 0, 4, 2, 0, 2, 6, 10, 14, 12, 10], [0, 0, 2, 2, 3, 1, 0, 4, 8, 12, 13, 11]]
```

```
Out[58]: (14, 'CGTATCG', 'CGTATCG')
```

The alignment score is: 14

The aligned sequences are:

'CGTATCG', 'CGTATCG'